

Kịch bản phần demo - bản dễ trình bày

Topic 8 - AI-Assisted DevOps - slide 27-33 - 7:00 - người nói: Huy

Bản tiếng Việt - rút gọn để trình bày, không phải tài liệu kỹ thuật

Đây là bản đã viết lại cho dễ nói. Mỗi bước chỉ còn ba phần: ý chính (một câu), bạn nói (script ngắn), và một câu dự phòng nếu bị hỏi sâu. Nếu cần bản đầy đủ, nhiều chi tiết kỹ thuật hơn (để chuẩn bị trả lời khó, hoặc để hiểu sâu hơn khi tập), xem file [DEMO_SCRIPT.md](#) (tiếng Anh) - đó là bản gốc, giữ nguyên toàn bộ chi tiết.

Có công cụ mới: `export_artifacts.py` - sinh ra file log thật, dài, mở được bằng Notepad/VS Code để cuộn cho cả lớp xem, cùng với kết quả sau khi cắt và kết quả model trả về. Xem hướng dẫn ở cuối tài liệu này.

Thời lượng

	Nội dung	Thời gian	Cộng dồn
Slide 27	Một demo - và điều gì là thật	0:50	0:50
LIVE	<code>py triage_demo.py</code> <code>--focus 5,7 --step</code>	3:10	4:00
Slide 29	Chuyển ý - một quyết định đã đo, không đoán	0:10	4:10
Slide 30	Kết quả: hòa	0:45	4:55
Slide 31	Vì sao vẫn chọn tool-based	0:45	5:40
Slide 32	Cấu trúc không phải ngữ nghĩa	0:40	6:20
Slide 33	Kết quả cuối / chốt phần demo	0:40	7:00

Chỉ có **đúng một lệnh** cần gõ trên sân khấu trong cả khối demo này.

PHẦN A - Trước khi chạy - 30 giây thành thật (0:50)

BẠN NÓI

Mình sẽ cho mọi người xem toàn bộ workflow, chạy trực tiếp, một lần từ đầu đến cuối.

Nhưng trước tiên, ba mươi giây thành thật.

Mọi thứ sắp diễn ra là code thật. Chín class thật. Kiểm tra chữ ký thật. Cắt log thật. Một lệnh gọi thật tới Claude - tụi mình trả tiền cho nó.

Cái chưa xây là phần hạ tầng mạng xung quanh - không có server đang chờ, không có tunnel, không có CI thật chạy. Tụi mình gọi thẳng hệ thống với một sự kiện đã lưu sẵn.

Bỏ đi phần không kiểm soát được. Giữ lại mọi phần đã tự viết.

Ghi nhớ nhanh - nếu bị hỏi "cái gì thật, cái gì không":

Thật	Chưa làm
9 class, chữ ký HMAC, bộ cắt log, lệnh gọi Claude, validate	HTTP server, tunnel, CI thật, lưu database, chống gửi trùng

PHẦN B - Chín bước - bản dễ nói

Lệnh (đã gõ sẵn, chưa Enter):

```
py triage_demo.py --focus 5,7 --step
```

Mỗi lần bấm Enter là qua một bước. Cứ theo đúng thứ tự bên dưới - không cần nhớ nhiều, mỗi bước chỉ có một ý và một câu nói.

Mở đầu - trước khi bấm Enter lần đầu

BẠN NÓI

Đây là sơ đồ kiến trúc lúc này. Chín bước. Bước tô đỏ - bước bảy - là AI duy nhất trong cả hệ thống.

BƯỚC 1 - Nhận webhook (0:10)

Ý chính: GitHub báo cho mình biết một lần chạy vừa xong, kèm nó fail hay pass.

BẠN NÓI

GitHub báo một lần chạy vừa xong. Run 8471023. Kết luận: fail. Trên pull request số một.

Nếu bị hỏi sâu: Mình không dùng thẳng dữ liệu gốc của GitHub - mình gói lại thành 8 trường thông tin của riêng mình. Vậy nếu GitHub đổi cấu trúc, chỉ một chỗ trong code cần sửa.

BƯỚC 2 - Kiểm tra chữ ký (0:15)

Ý chính: Ai cũng POST được lên endpoint này, nên phải kiểm tra "tin nhắn" có thật sự từ GitHub gửi không - giống soi chữ ký trên một tờ séc.

BẠN NÓI

Bất kỳ ai trên mạng cũng POST được tới đây. Nên trước khi làm gì khác, mình kiểm tra chữ ký số. Sai chữ ký, request bị chặn ngay - model không bao giờ thấy nó.

Nếu bị hỏi sâu: Mình dùng một hàm so sánh đặc biệt, không phải so sánh chuỗi thường, để tránh lộ thông tin qua thời gian xử lý.

BƯỚC 3 - Lọc sự kiện (0:10)

Ý chính: GitHub báo cho mình MỌI lần chạy, kể cả lần thành công. Mình chỉ giữ đúng loại: đã xong và bị fail.

BẠN NÓI

GitHub gửi mọi lần chạy, kể cả lần xanh. Ba điều kiện lọc. Chỉ giữ lần đỏ - vì mỗi lần gọi model đều tốn tiền.

Nếu bị hỏi sâu: Nếu bị lọc, mình trả lời "OK, bỏ qua" chứ không phải báo lỗi - để GitHub không gửi lại liên tục.

BƯỚC 4 - Tải log (0:15)

Ý chính: Tải log về. Đòi thật là một file ZIP; trong demo mình tạo sẵn một log y hệt thật, để không phụ thuộc mạng lúc trình bày.

BẠN NÓI

Và đây là log. Mười hai nghìn không trăm mười bảy dòng. Gần một megabyte. Không thể gửi nguyên vậy cho model. Bước tiếp theo mới là bước quan trọng.

Nếu bị hỏi sâu: Log được sinh ra theo một công thức cố định, nên số liệu luôn giống nhau mỗi lần chạy - không phải số được chọn cho đẹp.

BƯỚC 5 - Cắt log - bước quan trọng nhất, nói chậm lại (0:40)

Ý chính: Cắt mười hai nghìn dòng còn khoảng bốn mươi dòng - chỉ giữ phần liên quan tới lỗi.

Ba luật, dễ nhớ:

- Giữ 40 dòng cuối cùng (kết luận thường nằm ở đây)
- Giữ mọi dòng có từ khóa lỗi (10 từ khóa, ví dụ "ERROR", "FAILED")
- Giữ thêm 3 dòng quanh mỗi từ khóa, để có ngữ cảnh

BẠN NÓI

Mười hai nghìn dòng, mình giữ lại bốn mươi. Chín mươi chín phẩy bảy phần trăm bị bỏ.

Nhìn khối ở giữa màn hình - đây không phải bản tóm tắt, đây chính xác là văn bản gửi cho model.

Và đọc dòng cảnh báo đỏ ở cuối: "đây là bước dễ làm mất câu trả lời nhất." Nếu nguyên nhân thật nằm ở chỗ không có từ khóa lỗi, bộ lọc này sẽ bỏ sót, và model sẽ tự tin giải thích sai. Quân sẽ nói kỹ hơn ở phần rủi ro.

Nếu bị hỏi sâu: Vì sao không gửi nguyên cả log? Gửi nhiều hơn vừa tốn tiền, vừa khiến model khó tìm đúng chỗ hơn - giống đưa cả cuốn sách thay vì một trang.

BƯỚC 6 - Dựng prompt (0:10)

Ý chính: Ghép log đã cắt cùng vài thông tin (tên repo, mã lần chạy) thành một tin nhắn hoàn chỉnh.

BẠN NÓI

Tin nhắn đã dựng xong. Để ý - khuôn mẫu câu trả lời không nằm ở đây. Nó nằm ở một chỗ riêng. Đó là bước tiếp theo.

BƯỚC 7 - Gọi model - bước có AI (0:35)

Ý chính: Gọi Claude, và bắt buộc nó trả lời đúng khuôn (JSON) - không cho viết văn xuôi.

BẠN NÓI

Đây là bước AI duy nhất trong cả chín bước. Mình bắt buộc model trả lời qua một khuôn có sẵn - không được viết văn xuôi.

Và nhìn kết quả - nó trả về thẳng một object đã có cấu trúc, không cần mình tự bóc tách gì cả.

Khoảng một ngàn tám trăm token vào, hai trăm token ra. Tốn chưa tới một xu.

Nếu bị hỏi sâu: Nếu model không trả lời được hoặc API lỗi, demo tự động dùng một câu trả lời dự phòng đã lưu sẵn, để buổi trình bày không bị gián đoạn.

BƯỚC 8 - Kiểm tra lại kết quả (0:10)

Ý chính: Dù đã ép khuôn, mình vẫn kiểm tra lại - vì "đúng khuôn" không có nghĩa là "đúng nội dung".

BẠN NÓI

Sáu luật kiểm tra. Tất cả pass. Điều thú vị là - ba luật trong đó, hệ thống của Claude tự đảm bảo rồi. Ba luật còn lại thì không, ví dụ giới hạn sáu trăm ký tự chỉ là một dòng ghi chú chứ không phải luật thật sự. Nên mình vẫn phải tự kiểm tra.

BƯỚC 9 - Đóng gói và đăng (0:10)

Ý chính: Gói kết quả thành một comment gọn gàng, rồi đăng lên (hoặc ghi ra file trong demo).

BẠN NÓI

Kết quả được đóng gói thành một comment. Trong bot thật, nó đăng thẳng lên pull request. Ở đây mình ghi ra file để xem.

Và bot chỉ có đúng một quyền - đăng comment. Không merge, không xóa, không chạy lại gì cả. Đó là chủ đích.

Sau chín bước - xem kết quả (0:20)

BẠN NÓI

Đây là kết quả cuối - bốn thông tin, và một comment đọc được ngay.

Mười hai nghìn dòng đi vào. Một đoạn văn ngắn đi ra. Dưới bốn giây.

❑ Nếu còn dư giờ (tùy chọn, +0:20)

Quay xuống hỏi khán giả trước khi chạy:

Chọn một cái - dependency, syntax, hay timeout.

Chạy đúng cái họ chọn:

```
py triage_demo.py --log dependency
py triage_demo.py --log syntax
py triage_demo.py --log timeout
```

Năm giây, ra kết quả khác hẳn. Chứng minh không phải video quay sẵn.

PHẦN C - Kết quả thí nghiệm so sánh cơ chế (3:00 - chỉ dùng slide)

Không chạy lệnh nào ở phần này. Đây là lúc kể lại một kết quả đã đo sẵn từ trước, không phải diễn trực tiếp.

Slide 29 (0:10)

BẠN NÓI

Một chuyện nữa trước khi qua phần rủi ro. Bước bảy vừa dùng cách gọi là "tool-based". Có cách thứ hai gọi là "prompt-only". Tụi mình không đoán - đã đo cả hai, bốn mươi lần, từ trước.

Slide 30 - Hòa (0:45)

	runs	valid JSON	conformant	correct
prompt-only	20	100%	100%	100%
tool-based	20	100%	100%	100%

BẠN NÓI

Bốn log mẫu, năm lần mỗi cái, cả hai cách. Bốn mươi lệnh gọi.

Cách A - schema để trong tin nhắn, xin lịch sự. Kết quả trả về là một chuỗi, code của mình phải tự tìm JSON bên trong.

Cách B - chính là bước bảy vừa rồi. Model bị ép gọi qua một khuôn có sẵn, kết quả về đã được xử lý sẵn.

Và kết quả: cả hai đạt một trăm phần trăm, mọi lần. Hòa. Thí nghiệm tìm người thắng đã không tìm ra ai thắng.

Slide 31 - Vì sao vẫn chọn cách B (0:45)

BẠN NÓI

Ba lý do.

Một: hai mươi lần chưa chắc là một trăm phần trăm thật sự - với mẫu nhỏ vậy, cách A vẫn có thể thỉnh thoảng sai mà mình chưa gặp.

Hai - lý do chính: nếu cách B hỏng, đó là lỗi của Anthropic. Nếu cách A hỏng, đó là lỗi trong code của chính mình, và mình phải tự sửa.

Ba: mình mới test bốn log sạch sẽ. Những log dễ làm hỏng cách A nhất - mình chưa thử.

Nên không chọn B vì điểm cao hơn. Chọn vì khi nó hỏng, không phải lỗi của mình.

Slide 32 - Đúng khuôn không có nghĩa đúng ý (0:40)

BẠN NÓI

Một phát hiện nữa, đáng nhớ nhất.

Schema nói giải thích phải dưới sáu trăm ký tự. Nhưng câu đó chỉ là ghi chú, không phải luật thật sự.

Nên hệ thống chỉ đảm bảo có đủ trường thông tin. Không đảm bảo độ dài. Vì vậy bước tám - kiểm tra lại - vẫn phải tồn tại.

Bài học: ép khuôn chỉ đảm bảo hình dạng, không đảm bảo ý nghĩa.

Chuyển sang slide 33, khép lại 7 phút demo.

PHẦN D - Câu hỏi hay gặp - trả lời ngắn gọn

"**Chưa xây webhook thật à?**" -> Đúng, chưa. Phần đó chỉ hai mươi dòng code - gọi thẳng cái mình vừa demo. Thời gian dồn vào chín bước phía sau, vì rủi ro thiết kế nằm ở đó.

"**Vậy sao biết nó chạy được?**" -> Chưa biết chắc - đó là khoảng trống thật. Chưa test việc GitHub gửi lại, gửi trùng, gửi sai thứ tự.

"**Log là giả à?**" -> Sinh ra theo công thức cố định, nhưng lỗi ở cuối là lỗi pytest có thật. Sinh ra thay vì tải về để demo luôn giống nhau mỗi lần chạy.

"**Sao không chạy CI thật?**" -> Mất từ nửa phút tới vài phút, có thể xếp hàng chờ. Và nó chỉ test tốc độ của GitHub, không test code của mình.

"**Sao biết model không học thuộc một log?**" -> Đổi log khác - chạy `--log dependency` ngay.

"**Data đi đâu?**" -> Bốn mươi dòng đã cắt gửi qua API của Anthropic. Nếu repo có secret trong log, chưa giải quyết được vấn đề che giấu dữ liệu.

"Bot làm được gì với repo?" -> Chỉ đăng comment. Không merge, không xóa, không chạy lại. Có chủ đích - để chặn rủi ro nếu ai đó chèn lệnh độc hại vào log.

PHẦN E - Chuẩn bị & xử lý sự cố

Trước khi lên sân khấu

- [] `py triage_demo.py --offline` chạy sạch
- [] `py triage_demo.py --focus 5,7 --step` chạy được với API key thật
- [] Cỡ chữ terminal 20, cửa sổ gần full màn hình
- [] Lệnh gõ sẵn, chưa Enter - không gõ trước mặt khán giả

Nếu có sự cố - nói một câu, rồi chuyển tiếp:

Chuyện gì	Nói gì	Làm gì
Mất mạng / lỗi API	"Mạng ở đây bị chặn, đây là bản quay sẵn."	phát clip
Lỗi khác	"Để mình chạy chế độ offline."	<code>Ctrl-C -> --offline</code>
Treo > 10 giây	"Đường truyền chậm, đây là bản quay sẵn."	chuyển clip

PHẦN F - File log thật để show cho cả lớp

Đây là công cụ mới - sinh ra file thật, mở được bằng Notepad/VS Code, để bạn cuộn cho mọi người xem thay vì chỉ nhìn terminal chạy nhanh qua.

Cách chạy

```
cd D:\CI_Failure_Triage_Bot\demo
py export_artifacts.py
```

Có API key thì nó gọi Claude thật; không có thì tự động dùng bản dự phòng. Muốn chắc chắn không tốn tiền, thêm `--offline`. Muốn log khác, thêm `--log dependency` (hoặc `syntax`, `timeout`). Muốn sinh cả bốn loại một lần: `--all`.

Kết quả - nằm trong `demo\log_samples\test\` (hoặc theo tên loại log):

File	Là gì	Dùng để làm gì
<code>01_full_ci_log.txt</code>	log gốc, khoảng 12.000 dòng	mở ra, cuộn thật nhanh cho mọi người thấy nó dài cỡ nào
<code>02_trimmed_log_sent_to_model.txt</code>	~40 dòng còn lại sau khi cắt t	mở ngay sau đó - tương phản rõ rệt với file trên
<code>02b_full_prompt_sent.txt</code>	toàn bộ tin nhắn gửi cho model	dùng nếu ai hỏi "prompt thật trông thế nào"
<code>03_model_response.json</code>	kết quả model trả về	mở song song với comment bên dưới
<code>04_pr_comment.md</code>	comment cuối cùng	mở bằng trình xem Markdown nếu có, hoặc Notepad cũng đọc được
<code>00_SUMMARY.txt</code>	tóm tắt số liệu	để bạn liếc nhanh trước khi lên sân khấu

Gợi ý cách trình bày ba file này (nếu muốn tách riêng khỏi phần chạy live trong terminal, ví dụ lúc chuẩn bị hoặc lúc trả lời Q&A sâu):

1. Mở `01_full_ci_log.txt` trước - cuộn nhanh, để mọi người thấy độ dài.
2. Mở `02_trimmed_log_sent_to_model.txt` ngay sau - sự tương phản tự nó là luận điểm.
3. Mở `03_model_response.json` và `04_pr_comment.md` cạnh nhau - đầu vào máy móc, đầu ra con người đọc được.

Lưu ý: những file này dùng **đúng cùng một loại code** với bản chạy live trong terminal (`triage_demo.py`), nên số liệu luôn khớp nhau - không có chuyện terminal nói 40 dòng còn file lại nói 45 dòng.